CS6650 Assignment 1: SkiTrak Swipe Ingest Server and Load Client

Nikhil Pawar | October 2026

1 Git Repository

`https://github.com/aimsotrash/cs6650-assignments` (`/server`, `/client-part1`, `/client-part2`, `/results`, each with a README)

2 Design Document

2.1 Architecture

The server is one WAR (`skitrak.war`) deployed to Tomcat 9 on a t3.micro EC2 instance in us-west-2. It has the swipe endpoint, a health endpoint and the WebSocket leaderboard. The client sends swipes over HTTP and watches the leaderboards over WebSockets at the same time. For my final runs the client ran on a second EC2 instance in the same subnet: an m8azn.large (2 cores at 5.0 GHz) that I made from an image of my server, so it has the same setup. I also ran it on a c7g.large, a free-tier t3.micro and from home.

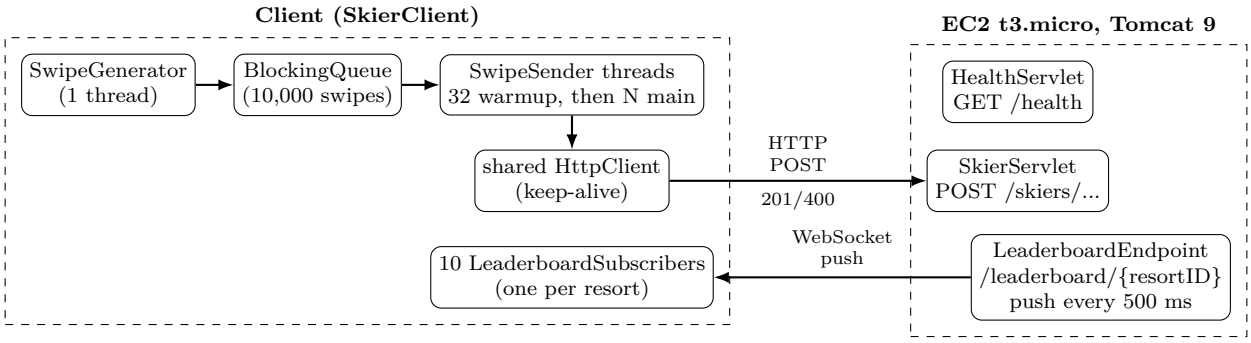


Figure 1: Architecture. HTTP for swipe ingest (request/response), WebSocket for the leaderboard (server push).

2.2 Major Classes

Server: `SkierServlet` is my Lab 2 servlet with stricter validation (IDs must be positive, `time` 1–360, `liftID` 1–40). It returns 201 or 400. I removed its per-request logging because logging 500K requests slows the server down. `HealthServlet` returns `{"message":"UP"}`. `LeaderboardEndpoint` maps each resortID to its open sessions, and one timer thread pushes a fake top 10 with a `serverTimestamp` every 500 ms. `AppListener` stops that timer on redeploy, otherwise every redeploy leaves an extra timer running. Models: `LiftRide`, `ResponseMsg`, `LeaderboardEntry`, `LeaderboardMessage`.

Client: `SkierClient` (`main`) runs the phases with one `SwipeGenerator` making `SwipeEvents`, the `SwipeSender` threads and 10 `LeaderboardSubscribers`. Part 2 adds `RequestRecord` (one CSV line) and `ThroughputChart`.

2.3 Threading Model

- **Generation and phases:** one generator thread puts all 500,000 swipes in a `LinkedBlockingQueue` with room for 10,000. It is much faster than the network, so the senders never wait for a swipe, and the fixed size keeps memory low. In the warmup, 32 threads send 1,000 swipes each and then end. The other 468,000 swipes are split between N main-phase threads (128 on the m8azn.large, 32 on the c7g.large, 64 on the t3.micro, 512 from home, see Section 3.7). Each phase has a `CountDownLatch`, so `main` times them separately.
- **HTTP:** all threads share one `java.net.http.HttpClient`, which keeps connections open and reuses them. With HTTP/1.1 a connection carries one request at a time, so there is at most one connection per sending thread (32 in the warmup, then N in the main phase). Tomcat closes a kept-alive connection after 100 requests (its default `maxKeepAliveRequests`), so a 500,000-swipe run opens about 5,000 connections. The client counts them from Tomcat's `Connection: close` responses and prints them with the retries and its own CPU use, plus the WebSocket connections and reconnections.
- **Errors:** a failed send is retried up to 5 times, waiting 100, 200, 400, 800 and 1600 ms. Swipes that still fail are counted as failed. The counters are `AtomicInteger`s.

- **Part 2 metrics:** each sender times its requests with `System.nanoTime()`, the monotonic clock from Week 4, so latency isn't rounded to whole ms (the CSV timestamp stays on the wall clock). Each sender keeps its own list of `RequestRecord`s (no locking while sending), and at the end one writer writes them all to the CSV. This is the lesson from Lab 2 Exercise 3, where several writers on one file produced broken lines.

2.4 WebSocket Connection Management

The client opens one subscriber for each of the 10 resorts before the warmup and keeps them open for the whole run. They share one Tyrus `WebSocketContainer`. Each subscriber records its connect time, counts updates, and saves *receive time* − *serverTimestamp* as the push latency of each update. If a connection closes mid-run, `onClose` starts a thread that reconnects (up to 5 tries, 1–5 s apart). When I redeployed the server during a run, all 10 subscribers reconnected and the 128 swipes that failed during the redeploy succeeded on retry, so 0 swipes failed. On the server, `@OnClose` and `@OnError` remove the session so the map doesn't leak.

2.5 Little's Law

$N = X \times R$, so $X = N/R$: N is the number of requests in flight (one per sender thread), R is the response time (round trip) of one swipe and X is the throughput (swipes per second). I measured R with 200 POSTs on one kept-alive connection after a 3,000-request warmup: 0.29 ms from my m8azn.large client, 0.49 ms from my c7g.large and 0.58 ms from my t3.micro in the server's subnet, and 16 ms from my home server over Ethernet and 19 ms over Wi-Fi. **Connection overhead:** Tomcat closes each connection after 100 requests, so 1 request in 100 also opens a new connection. Inside the subnet that costs about one extra round trip, about 1% of R. From home a new connection took 138 ms instead of about 19 ms, which adds about 1.2 ms per request on average (6%). Neither explains the gap. My calculations are in Figure 2.

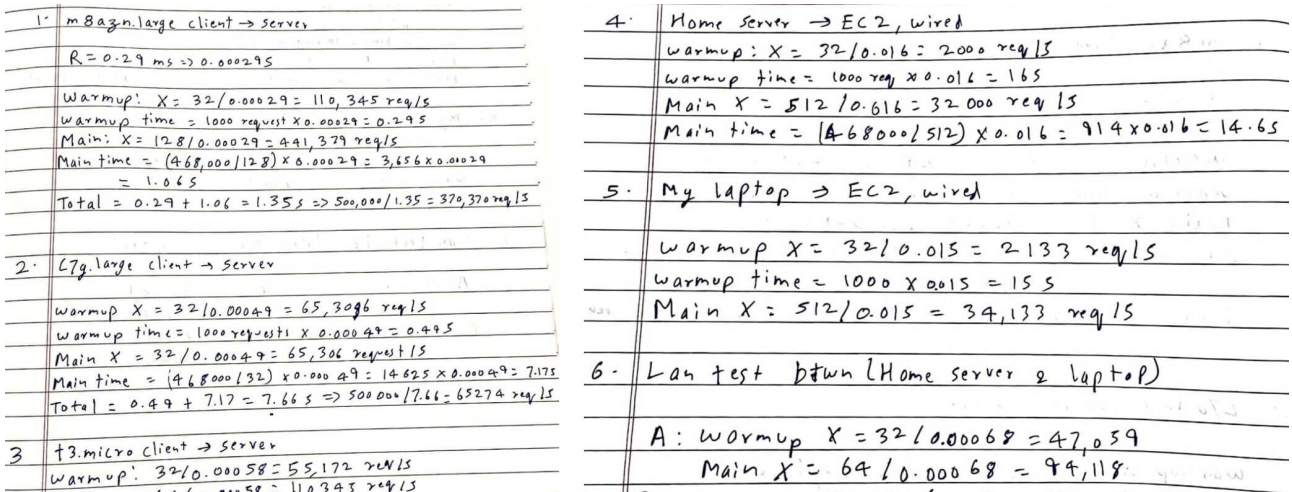


Figure 2: My Little's Law calculations and predictions.

Measured vs predicted: for my final m8azn.large run, Little's Law predicts 128/0.00029 = 441,379 swipes/s, but I measured 22,147 swipes/s in the main phase. My client is not the limit any more (it was only 53% busy); the t3.micro server is (95% busy in the same setup). I predicted that ceiling from a 32-thread run, where the server was 80% busy at 16,741 swipes/s: that is $2 \times 0.80/16{,}741 = 96$ μs of server CPU per swipe, so at 100% the server can do about 2/0.000096 = 20,900 swipes/s, and with 128 threads I measured 22,147 (6% above it: with more connections the server spent a little less CPU per swipe, 87 instead of 96 μs). Under load the requests wait at the server, so R grew from 0.29 ms to $128/X = 5.8$ ms, and Part 2 agrees: 21,311 × 0.00570 s = 121, about 128 threads. With my c7g.large client (10/14) the client was the limit instead (88–99% CPU, 14,290 swipes/s). From home the warmup matched the prediction (1,840/s vs 2,000/s), but the main phase was limited by my home upload (about 53–61 Mbit/s).

3 Test Results

3.1 Final Runs (500,000 swipes each, 0 failed)

Throughput means swipes per second: how many swipe POSTs got a 201 back each second (successful swipes ÷ time). For example, my final Part 1 run sent 500,000 swipes in 23.07 s, so its overall throughput is 500,000 / 23.07 = 21,671 swipes/s. In the tables, "/s" means swipes per second. Both clients ran on EC2 in the server's subnet and used its private IP.

	Part 1 (final) m8azn.large client	**Part 2 (final)** m8azn.large client	Part 1 (10/14) c7g.large client	Part 2 (10/14) c7g.large client
Main phase threads	128	128	32	32
Round trip (no load)	0.29 ms	0.29 ms	0.49 ms	0.49 ms
Warmup (32 × 1000): time, throughput	1.9 s, 16,486/s	2.2 s, 14,856/s	6.3 s, 5,076/s	5.3 s, 6,062/s
Main phase: time, throughput	21.1 s, 22,147/s	22.0 s, 21,311/s	32.7 s, 14,290/s	35.2 s, 13,306/s
Total time, overall throughput	23.1 s, **21,671/s**	24.1 s, 20,734/s	39.1 s, 12,803/s	40.4 s, 12,361/s
Retries / failed swipes	0 / 0	0 / 0	0 / 0	0 / 0
HTTP connections reopened	4,976	4,985	not counted	not counted
Client CPU busy (main phase)	53%, 48 µs/swipe	49%, 46 µs/swipe	88–99%	88–99%
Ingest latency mean/median/p95/p99	–	5.699 / 5.418 / 9.241 / 13.681 ms	–	2.57 / 2 / 5 / 10 ms
Ingest latency min / max	–	0.226 / 103.878 ms	–	0 / 169 ms
Throughput per resort	–	2,056–2,092/s	–	1,231–1,239/s
Leaderboard updates delivered	470	480	780	811
Push latency (Part 1: mean, max)	1 ms, max 20 ms	–	2 ms, max 28 ms	–
Push latency mean/median/p95/p99	–	1.59 / 1 / 3 / 24 ms	–	2.23 / 2 / 6 / 12 ms
WebSocket connect time (avg)	10 ms	13 ms	23 ms	22 ms

With the m8azn.large client the same ingest code went from 12,803 to 21,671 swipes/s. It has 2 real cores at 5.0 GHz (the c7g.large has 2 at 2.6 GHz and the t3.micro 1 core with 2 hyperthreads), so my client stopped being the limit and the server became it. On 10/14 the latencies were rounded to whole ms; now they come from `System.nanoTime()`.

Clock skew: push latency compares the client clock with the server clock. All my EC2 instances sync to the Amazon Time Sync Service (the server was 6 ns off when I checked), so these push latencies are real. From home, my Windows clock kept drifting even after I synced it, so the push latency came out wrong (−130 ms one day and +635 ms another day). That is skew, not a bug, and it's one reason my final runs are on EC2.

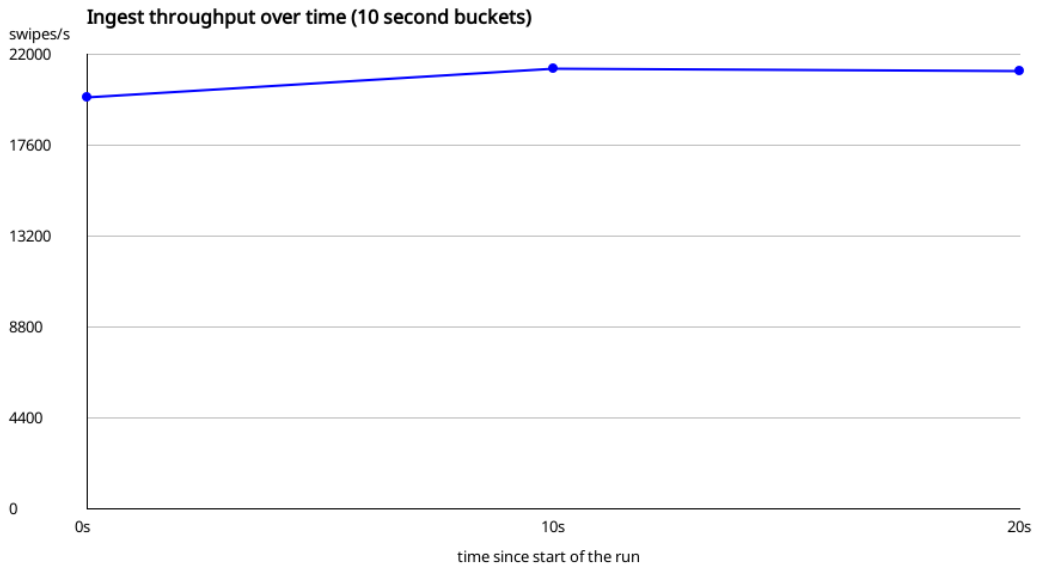
3.2 Part 1 Output (final, m8azn.large client)

```
[ec2-user@ip-172-31-38-155 ~]$ java -jar client-part1.jar http://172.31.36.18:8080/skitrak 128 | tee part1_output.txt
===== Ingest =====
Server: http://172.31.36.18:8080/skitrak
Successful swipes: 500000
Failed swipes: 0
Warmup (32 threads x 1000): 1941 ms, 16486 swipes/s
Main phase (128 threads): 21131 ms, 22147 swipes/s
Total run time: 23072 ms
Overall throughput: 21671 swipes/s
HTTP: 1 shared HttpClient (keep-alive), up to 128 connections open at once, 4976 reopened after the server closed them (Connection: close), 0 retries
Client CPU (main phase): 53% busy on 2 cores, 48 us of CPU per swipe → limit at 100% CPU about 41,804 swipes/s
===== Leaderboard =====
Resort 1: 47 updates, connect time 81 ms
Resort 2: 47 updates, connect time 3 ms
Resort 3: 47 updates, connect time 6 ms
Resort 4: 47 updates, connect time 2 ms
Resort 5: 47 updates, connect time 2 ms
Resort 6: 47 updates, connect time 2 ms
Resort 7: 47 updates, connect time 2 ms
Resort 8: 47 updates, connect time 2 ms
Resort 9: 47 updates, connect time 2 ms
Resort 10: 47 updates, connect time 3 ms
WebSocket connections: 10 (reconnections: 0)
Average WebSocket connect time: 10 ms
Push latency: mean 1 ms, max 20 ms (470 updates)
[ec2-user@ip-172-31-38-155 ~]$
```

3.3 Part 2 Output (final, m8azn.large client)

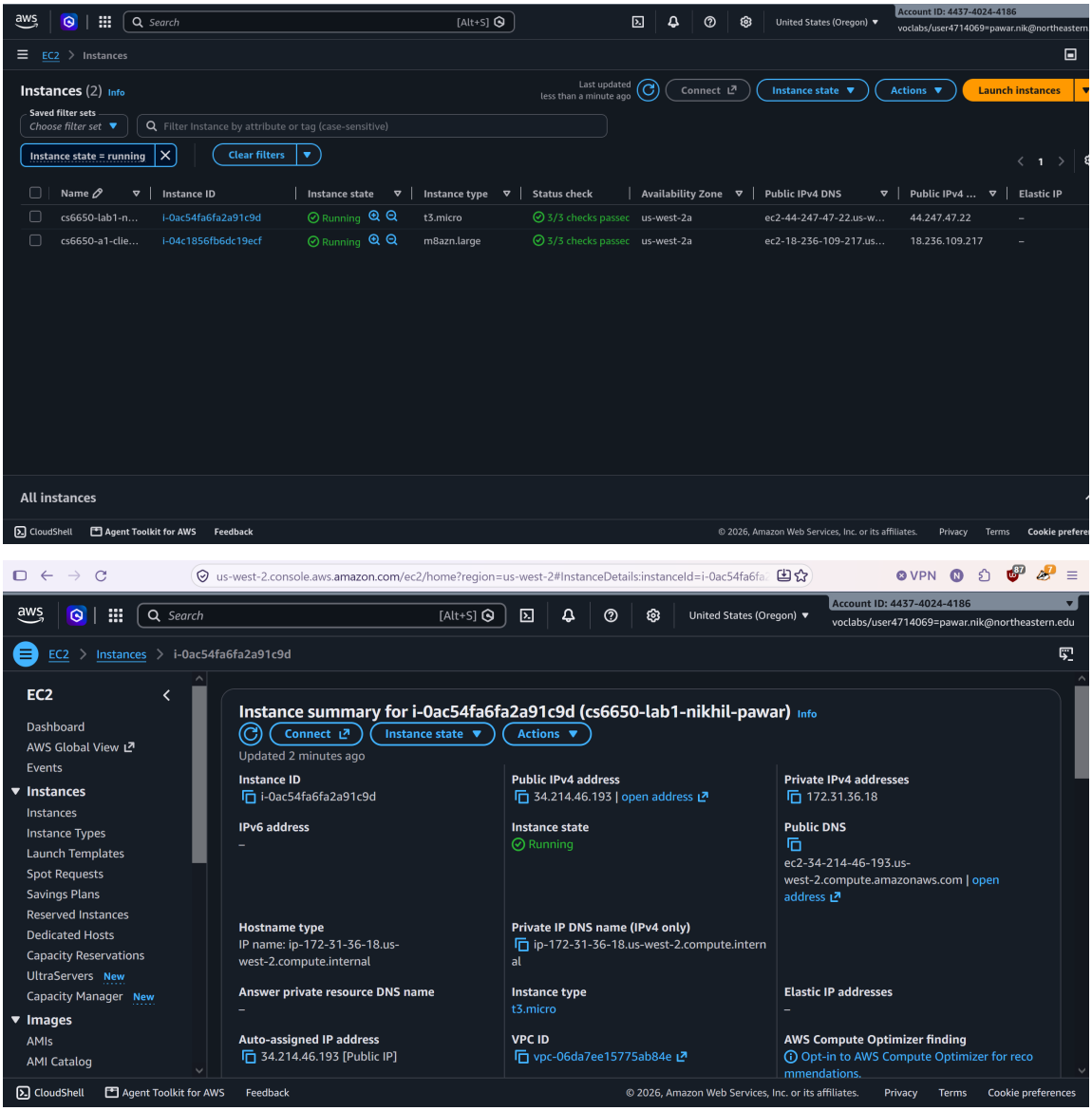
```
[ec2-user@ip-172-31-38-155 ~]$ java -jar client-part2.jar http://172.31.36.18:8080/skitrak 128 500000 results_final | tee part2_output.txt
===== Ingest =====
Server: http://172.31.36.18:8080/skitrak
Successful swipes: 500000
Failed swipes: 0
Warmup (32 threads x 1000): 2154 ms, 14856 swipes/s
Main phase (128 threads): 21960 ms, 21311 swipes/s
Total run time: 24114 ms
Overall throughput: 20734 swipes/s
HTTP: 1 shared HttpClient (keep-alive), up to 128 connections open at once, 4985 reopened after the server closed them (Connection: close), 0 retries
Client CPU (main phase): 49% busy on 2 cores, 46 us of CPU per swipe → limit at 100% CPU about 43,193 swipes/s
===== Ingest latency (ms) =====
Requests: 500000
Mean: 5.699
Median: 5.418
p95: 9.241
p99: 13.681
Min: 0.226
Max: 103.878
===== Throughput per resort =====
Resort 1: 50003 swipes, 2073 swipes/s
Resort 2: 50169 swipes, 2080 swipes/s
Resort 3: 49599 swipes, 2056 swipes/s
Resort 4: 49808 swipes, 2065 swipes/s
Resort 5: 49974 swipes, 2072 swipes/s
Resort 6: 50464 swipes, 2092 swipes/s
Resort 7: 50162 swipes, 2080 swipes/s
Resort 8: 50178 swipes, 2080 swipes/s
Resort 9: 49831 swipes, 2066 swipes/s
Resort 10: 49812 swipes, 2065 swipes/s
===== Leaderboard =====
Resort 1: 48 updates, connect time 79 ms
Resort 2: 48 updates, connect time 9 ms
Resort 3: 48 updates, connect time 17 ms
Resort 4: 48 updates, connect time 6 ms
Resort 5: 48 updates, connect time 4 ms
Resort 6: 48 updates, connect time 5 ms
Resort 7: 48 updates, connect time 4 ms
Resort 8: 48 updates, connect time 6 ms
Resort 9: 48 updates, connect time 2 ms
Resort 10: 48 updates, connect time 4 ms
Total leaderboard updates delivered: 480
WebSocket connections: 10 (reconnections: 0)
Average WebSocket connect time: 13 ms
===== Push latency (receive time - serverTimestamp, ms) =====
Mean: 1.59
Median: 1
p95: 3
p99: 24
Files: results_final/swipe_latencies.csv, results_final/throughput.csv, results_final/throughput.png
[ec2-user@ip-172-31-38-155 ~]$
```

3.4 Throughput Over Time (final Part 2 run, 10 second buckets)



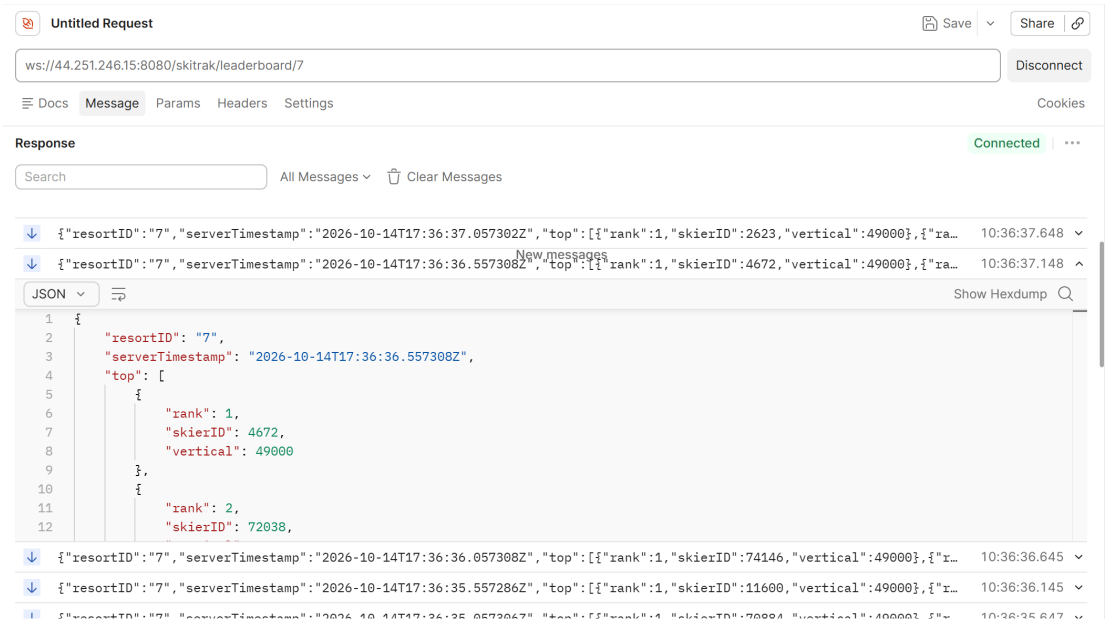
The run is short (24.11 s), so the chart has 3 points: 19,942 / 21,335 / 21,202 swipes/s. The first bucket includes the warmup (32 threads, cold JVMs); after that the throughput stays flat because the server is at its limit. On 10/14, with the c7g.large client, throughput rose from about 7,400 to 16,100 swipes/s over a 40 s run while the client warmed up (`results/earlier_runs/oct14_c7g_t3/`).

3.5 EC2 Deployment



The first screenshot shows the server and the m8azn.large client running on the day of my final runs. The second one shows the server's details; its public IP changes each time the lab starts, but the private IP 172.31.36.18 stays the same. The WAR deployed on the server has the same bytecode as the server code in this repo.

3.6 WebSocket Subscriber Receiving Leaderboard Pushes



Postman connected to `/leaderboard/7`, receiving a synthetic top 10 with a `serverTimestamp` every 500 ms.

3.7 Experiments: Where Should the Client Run?

Where the client runs, and on what, decides what limits the throughput. These are my best Part 1 runs in five other setups; every run is in `results/experiment_client_location.txt`.

Client → server	Round trip	Threads	Main phase throughput	Overall throughput
Client EC2, c7g.large → my t3.micro server, best of 7 runs	0.49 ms	32	14,623/s	**13,605/s**
My home server (Lenovo, i7-8650U), wired → EC2	16 ms	512	21,968/s	**12,922/s**
My laptop (Razer Blade 14, Ryzen 9 5900HX), wired → EC2	15–38 ms	512	18,242/s	**11,979/s**
LAN test: my laptop → my home server (Tomcat, not EC2)	0.94 ms	64	41,393/s	**38,299/s**
Client EC2, m8azn.large → m8azn.large server (Section 3.8)	0.11 ms	64	49,476/s	**45,330/s**

The round trips inside AWS were measured with Java after a 3,000-request warmup. The others were measured with 200 requests from home or on the LAN.

- **c7g.large client (my final setup on 10/14):** Learner Lab only allows instances up to `.large`. On Intel a `.large` is 1 core with 2 hyperthreads, but a Graviton c7g.large has 2 real cores, so I tried one in the server's subnet. Its 7 runs over two days were between 12,409 and 13,605 swipes/s. Its CPU was 88–99% busy, so the client was the limit, while the server was about 75% busy. The same jar on Java 21 was a bit slower (12,568–12,774 swipes/s) because its warmup was slower.
- **My home server (Lenovo) as the client:** its best run was wired: 12,922 swipes/s with 512 threads. From home the round trip (16 ms) sets the warmup (1,840/s, close to the 2,000/s Little's Law predicts) and my home upload (about 61 Mbit/s, full) sets the main phase, not the client CPU (63% busy). Ethernet cut the round trip from 19 to 16 ms and made the whole run about 15% faster than on Wi-Fi (11,226 → 12,922 swipes/s). On another day the round trip was 25 ms and the same run gave only 11,246–11,573 swipes/s.
- **My laptop (Razer) as the client:** its best run was 11,979 swipes/s with 512 threads, wired. Its faster CPU did not help: in my first test it got about the same as my home server (11,174/s) with 66% of its CPU idle, which also points to my home upload (59 Mbit/s). Its round trip changed that evening (15 ms in the first test, 38 ms before the best run, during a video call), and it had 4–57 retries per run that I couldn't explain (Ethernet only, no link errors).
- **LAN test (technically a cheat):** to check the upload idea, I took EC2 out. I ran Tomcat on one of my machines and the client on the other, both wired to my home router (1 Gbit/s). My laptop as the client and my home server as the server gave **38,299 swipes/s** (500,000 swipes in 13.1 s), and the other way around gave 27,495 swipes/s. That is 1.3–1.8 times my final run. It is faster because nothing goes over the internet: the round trip is about 17 times shorter than from home (0.94 ms vs 16 ms), and the 1 Gbit/s link is about 16 times my home upload. The limit moved to CPU instead (my home server was 81% busy as the server and 75% busy as the client). The LAN run is not a valid Part 1 result because the server is supposed to run on EC2, and my home server has more CPU resources than the t3.micro. On 10/15 the t3.micro topped out at about 22,000 swipes/s (95% CPU), so it could not keep up with 38,000 swipes/s. My final Part 1 result is 21,671 swipes/s. The LAN test shows how much faster the same code runs when the network is not the bottleneck.

3.8 Experiment: m8azn.large Client and m8azn.large Server

With the m8azn.large client my t3.micro server was the limit (95% CPU). To see how fast my client can go when the server is not the limit, I launched two m8azn.large instances (2 real AMD cores at 5.0 GHz each) from an image of my server, in the same subnet: one runs Tomcat with the same WAR, the other runs the same client. This is only an experiment, because the assignment's server has to be a free-tier instance.

- **Round trip:** 0.11 ms median (0.15 ms mean), vs 0.29 ms to the t3.micro.
- **Thread sweep (200,000 swipes, main phase):** 32 threads 38,700/s, 64 threads 39,289/s, 128 threads 32,754/s, 256 threads 31,437/s. More threads only add client CPU (48 μs per swipe at 32 threads, 63 at 256).
- **Best run (500,000 swipes, 64 threads):** warmup 20,369/s, main phase 49,476/s, overall **45,330 swipes/s**, 0 failed, 0 retries. The client was 93% busy (38 μs of CPU per swipe) and the server about 53% busy, so my client is the limit again. Four 500,000-swipe runs gave 42,276–45,330 swipes/s.
- **What it shows:** the same client code more than doubles its throughput when the server is not the limit (45,330 vs 21,671 swipes/s), and beats my LAN test (38,299 swipes/s).